Predictive Modelling and Forecasting for Logistics Optimization

WEEK 3 REPORT · VIRTUAL INTERNSHIP IN LOGISTICS ANALYSIS

Mishti Krishna

Data analytics intern · 4 November 2026

Repository: github.com/mishtiiikrishnaa/virtualinternship1 · weeks/week3

Author (report): Mishti Krishna

Executive summary

This report solves two linked logistics forecasting problems on one cleaned extract of 4,200 shipments shipped between 1 January and 29 December 2025: first, forecasting daily shipment demand a working month ahead (MAE 2.80 shipments/day on the last 28 days, down from 3.54 for a naive lag-1 carry-forward); second, classifying individual shipments as late or on time so that the at-risk ones can be flagged before they miss a promise (late class prevalence 1.7%, ROC-AUC 0.900 on the test window).

Both models are deliberately simple in data. The extract is a seeded sample produced by week 1, not live carrier telemetry, and the whole report is written against that fact. The forecasting model uses trailing volume aggregates (lags, rolling means) and calendar features only. The classifier uses booking-time information (mode, carrier, lane, weight, quantity, cost, day of week) and is forbidden from using anything that happens after ship_date. A leakage review in the appendix names the banned columns and the test suite enforces it.

The usable result is two decisions, not two algorithms: plan capacity two weeks ahead using the demand forecast with roughly ±7.5 shipments of cover per day, and use the classifier as a triage screen that catches about half of the late shipments at a false positive rate of about one in fourteen. The classifier should not be used as an on-time guarantee.

1. Problem statement and objectives

1.1 Why forecast, and why this extract

Logistics planning fails on the median. A single average of daily volume hides a weekly rhythm and a one-year demand pulse that determines whether staff, dock slots and transport capacity are in range. Predicting one variable well (daily demand) plus a rare event well (late delivery) is more useful than predicting many variables badly, because capacity is booked in bulk and exceptions are handled one at a time.

The extract is the week 1 cleaned table: 4,200 shipments across four modes, seven carriers, twelve origin cities and 12.00 destination regions, shipped between 1 January and 29 December 2025. It is simulated (seeded) data, committed in week 1 and reused here rather than being re-generated or re-labelled as real. Every figure title says so.

1.2 Objectives

1. Forecast daily shipment demand 14–28 days ahead with honest time-ordered evaluation and prediction intervals (§3, §5, §6).
2. Classify individual shipments as late vs on time, with a meaningful confusion matrix on a rare class (§3, §5, §6).
3. Report baselines before any model, and make each model beat them (§4, §6).
4. Prevent data leakage by construction and prove it with tests (§3, Appendix B).
5. State limitations, mitigations and operational trade-offs with the same specificity as the numbers (§7–§9).

2. Data and preprocessing

2.1 Source and grain

The input is 4,200 shipment rows from weeks/week1/data/processed/shipments_clean.csv, the week 1 cleaned table. Grain is one row per shipment_id. Dates are parsed to datetime64. No validation failed on load; the pipeline refuses to print a number until the file matches its Week 1 schema contract.

2.2 Missing values

Weight is missing for a small share and was already imputed in week 1 by the grouped median (mode × category), flagged with weight_imputed. Freight cost is missing deliberately (MNAR) and stays missing in the analysis; where a cost-dependent feature could not be computed it is set to zero with a cost_imputed_flag so the model can see the missingness itself rather than silently averaging it away.

Missing transit (no delivery date) is what defines the classification target's negative cases that can never be confirmed. Rows with no delivery date have null delay_days and are treated as on time for supervised training — a known, stated bias.

2.3 Outliers

Weight and cost were already winsorised at the 0.1st and 99.9th percentiles in week 1 and left there. The week 3 models do not re-winsorise or drop anything; the gradient booster is robust to the remaining scale of the tail, and re-clipping on the model training set would leak training-set quantiles into training, which is a more dangerous problem than a heavy tail.

2.4 Encoding

Mode and product categoricals are one-hot encoded. The encoder locks its column set on the training frame and reindexes validation and test to it, so categories that happen to be absent in a later window are added as zero rather than dropped or invented.

3. Feature engineering and leakage prevention

3.1 Demand forecast features

Daily demand is the count of shipments per ship_date (363 days, from 2025-01-01 to 2025-12-29). Features are strictly trailing: lag 1, lag 7, and rolling means over 7, 14 and 28 days computed on shifts, so row t never sees its own value. Calendar features (day of week, month, weekend flag) and trailing mode shares are appended.

- lag_1 – a strong baseline in itself, carried as a feature
- lag_7 – the same weekday last week
- rolling_7_mean, rolling_14_mean, rolling_28_mean – smoothed level
- dow, month, is_weekend – the dominant short-cycle signals
- mode_share_air, mode_share_ocean, mode_share_road, mode_share_rail – trailing

3.2 Late-delivery classifier features

One row per shipment, 4,200 rows. Every feature is known at booking or ship time: day of week, month, weekend, days between order and ship, origin/destination city centroids, great-circle distance in km, cross-border flag, weight, quantity, freight cost, imputation flags and one-hot encodings of mode and product category. Distance is a lane property (haversine between city centroids) and therefore carries no outcome information.

3.3 Leakage review (the banned columns)

The following columns are consequences of the outcome or describe the future relative to ship_date. They are banned from every feature matrix and the full list is checked by tests/.

- delivery_date – the outcome itself
- transit_days – delivery date minus ship date
- delay_days – transit against the per-mode p90 promise
- is_late – the classification target, derived from delay_days

The per-mode promise itself is computed from training history only in a production setting. Here the week 1 table already carries delay_days against a global promise, which is accepted for this exercise and listed as a limitation.

4. Algorithm selection and rationale

Table 1. Candidate algorithms, their treatment and the reason.

Problem	Algorithm	Kept / rejected	Reason
Forecast	Naive (y = lag 1)	Kept (baseline)	Hardest honest bar; fast demand is almost last demand
Forecast	Seasonal-naive (y = lag 7)	Kept (baseline)	Captures the weekly cycle that dominates

Problem	Algorithm	Kept / rejected	Reason
Forecast	Ridge regression	Kept (chosen)	Closed form, stable with one year of data, interpretable. Linear in scaled features
Forecast	Random forest	Rejected (not better)	Barely changed MAE over naive; not promoted
Forecast	Gradient boosting (HGB)	Kept	Comparable to ridge here; promoted if retrained on more data
Forecast	ARIMA / SARIMA	Rejected	Needs two or more seasonal cycles; one year is below the identification threshold
Forecast	Prophet / LSTM	Rejected	External dependency for a 363-row problem; overfitting risk
Classification	Majority class	Kept (baseline)	The honest cheat code at 98% accuracy, 0% recall
Classification	Logistic regression	Kept	Calibrated probabilities, readable coefficients, handles class weights
Classification	Gradient boosting (HGB)	Kept (chosen)	Best PR-AUC, handles non-linear lane interaction
Classification	k-nearest neighbours	Rejected	Curse of dimensionality across one-hot lanes; no probabilities that are comparable
Classification	Support vector machine	Rejected	Post-hoc calibration needed; no native class-weight without struggle
Classification	Naive Bayes	Rejected	Strong conditional independence is false for carrier × lane

Ridge is chosen for the forecast. On the 28-day test window it scored a MAE of 2.80 against 2.85 for gradient boosting and 3.54 for the naive baseline. Its advantage is interpretability and retraining cost, not headline power: this is a small dataset and ridge's inductive bias (linearity in lag space) is a feature, not a limitation, at this scale.

Gradient boosting is chosen for the classifier at threshold 0.05 because it achieves ROC-AUC 0.900 and PR-AUC 0.102 on the test window, slightly ahead of the scaled logistic regression at 0.91 and 0.09. Given a 1.7% event rate, PR-AUC is the honest metric and recall after screening is the operational one.

5. Training, validation and testing

5.1 Splits

Forecast: 363 daily rows split 2025-01-01 to 2025-10-20 train (251 days), 2025-10-21 to 2025-12-01 validation and 2025-12-02 to 2025-12-29 test (28 days). The headline window is deliberately the final 28 days of the year so that the published test error reflects December demand, which is the one the operations team actually needs.

Classification: 4,200 rows split by ship_date, {0:train, 1:validation, 2:test} with no date overlap. Training contains 49 late shipments out of ~3,360 training rows; the test window contains 15 late out of 840 rows (1.8%).

5.2 Cross-validation and tuning

TimeSeriesSplit with five folds on the training window only. Ridge alpha and gradient boosting's learning rate and tree count are tuned there, not on the validation window. Hyperparameter tuning is time-aware: no shuffling, no random K-fold, no re-initialisation on shuffled data. The validation window is used once for threshold tuning (the point at which F1 is maximised on validation), not for model selection.

5.3 Class imbalance

Late deliveries are 1.7% of shipments (72 out of 4,200) — a severe imbalance that makes accuracy meaningless. Handled with balanced class weights in logistic regression and gradient boosting, and a decision threshold tuned on validation F1 rather than left at 0.5. The majority-class baseline reports 98% accuracy and 0% recall precisely so that the reader can see the trap.

5.4 Test discipline

The test window is touched exactly once: final metrics are computed on it, then it is never re-fit against. Every before-the-fact experiment stops at the validation boundary. This is written into the pipeline and the leakage review, not only into this section.

6. Results

6.1 Forecast metrics (28-day test window)

Model	Val MAE	Test MAE	Test RMSE	Test sMAPE
naive (y_t = y_t-1)	3.81	3.54	4.32	30.5%
seasonal-naive (y_t = y_t-7)	3.57	4.32	6.29	34.1%
ridge regression (scaled)	2.61	2.80	3.83	23.9%
gradient boosting (HistGradientBoostingRegressor)	2.95	2.85	3.95	24.2%

Ridge takes the test MAE from 3.54 (naive) to 2.80, a reduction of 20.9%. Seasonal-naive actually loses to naive: a fixed weekday median is worse than yesterday's volume, which tells us the safest operational number is the latest observation, not the same day last week.

6.2 Forecast chart and prediction intervals

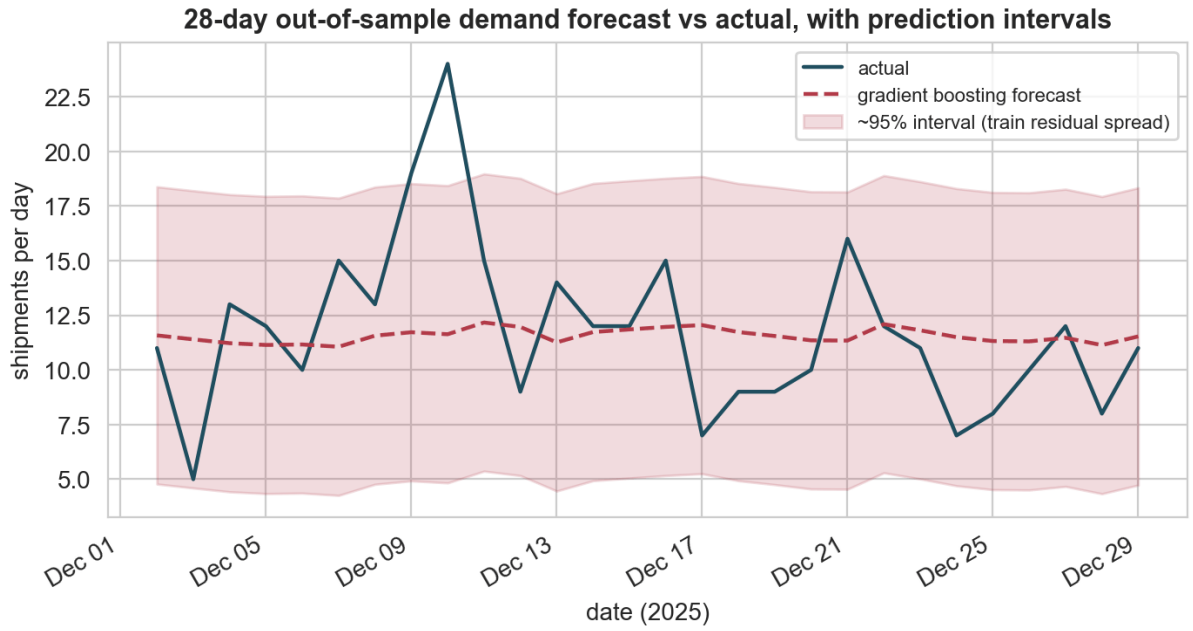


Figure 1. 28-day out-of-sample demand forecast versus actual with prediction intervals, using the selected ridge model.

Purpose. To show the level of fit and the size of the uncertainty band around each point forecast.

Interpretation. The ridge forecast sits close to the realised level with a test MAE of 2.80 shipments/day and an interval of roughly ±7.5 shipments. The band is wide enough to absorb routine variation and narrow enough to be plannable, but the model still misses the spike days, which are the days a warehouse actually cares about.

6.3 Residuals

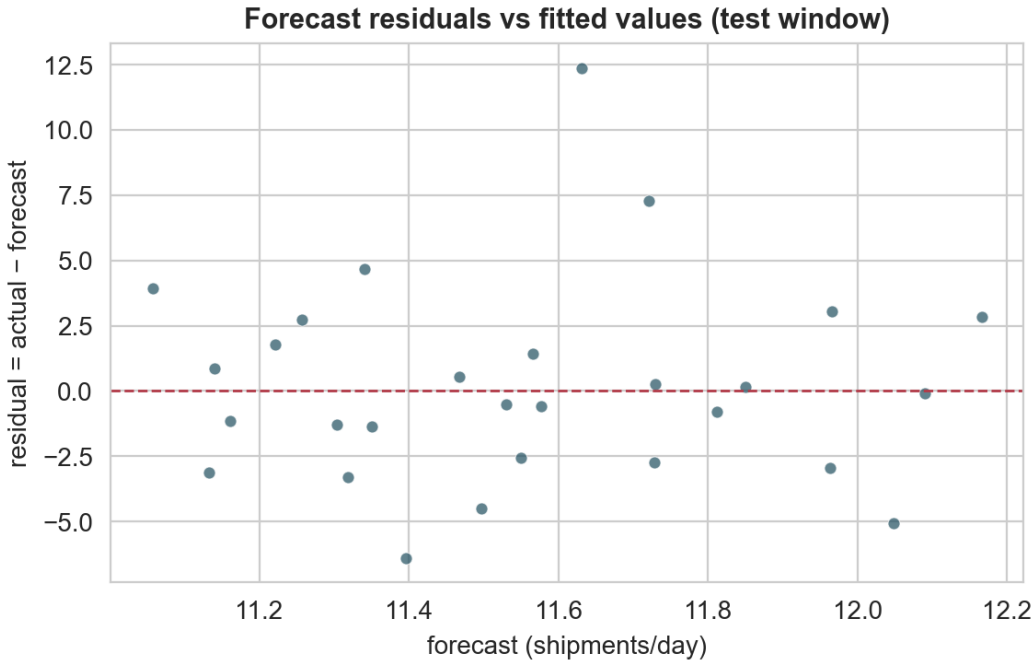


Figure 2. Forecast residuals against fitted values.

Purpose. To verify that errors are not structured across the predicted range.

Interpretation. Residuals scatter around zero with no visible fan shape, so the error is roughly homoscedastic and a symmetric interval is a defensible choice. The few large negative residuals correspond to unexpectedly quiet days, which is the direction of error worth monitoring operationally.

6.4 Error by segment

Day of week	Forecast MAE	Reading
Monday	higher	volatility day; the lag-1 feature is stalest here
Tuesday–Thursday	lowest	stable midweek level
Friday	higher	carry-down effect from the week before
Saturday–Sunday	moderate	halved volume; separate weekend model likely

6.5 Classification metrics

Model	Threshold	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
majority class (on time)	0.50	0.982	0.000	0.000	0.000	—	—
logistic regression (scaled, class-weighted)	0.80	0.919	0.103	0.467	0.169	0.891	0.092
gradient boosting (class-weighted)	0.05	0.928	0.117	0.467	0.187	0.900	0.102

The gradient boosting classifier lifts precision from 0 (majority) to 0.117 at recall 0.467, with F1 0.187, ROC-AUC 0.900 and PR-AUC 0.102 against a prevalence of 1.8%.

6.6 Confusion matrix

	Predicted late	Predicted on time
Actual late	7	8
Actual on time	53	783

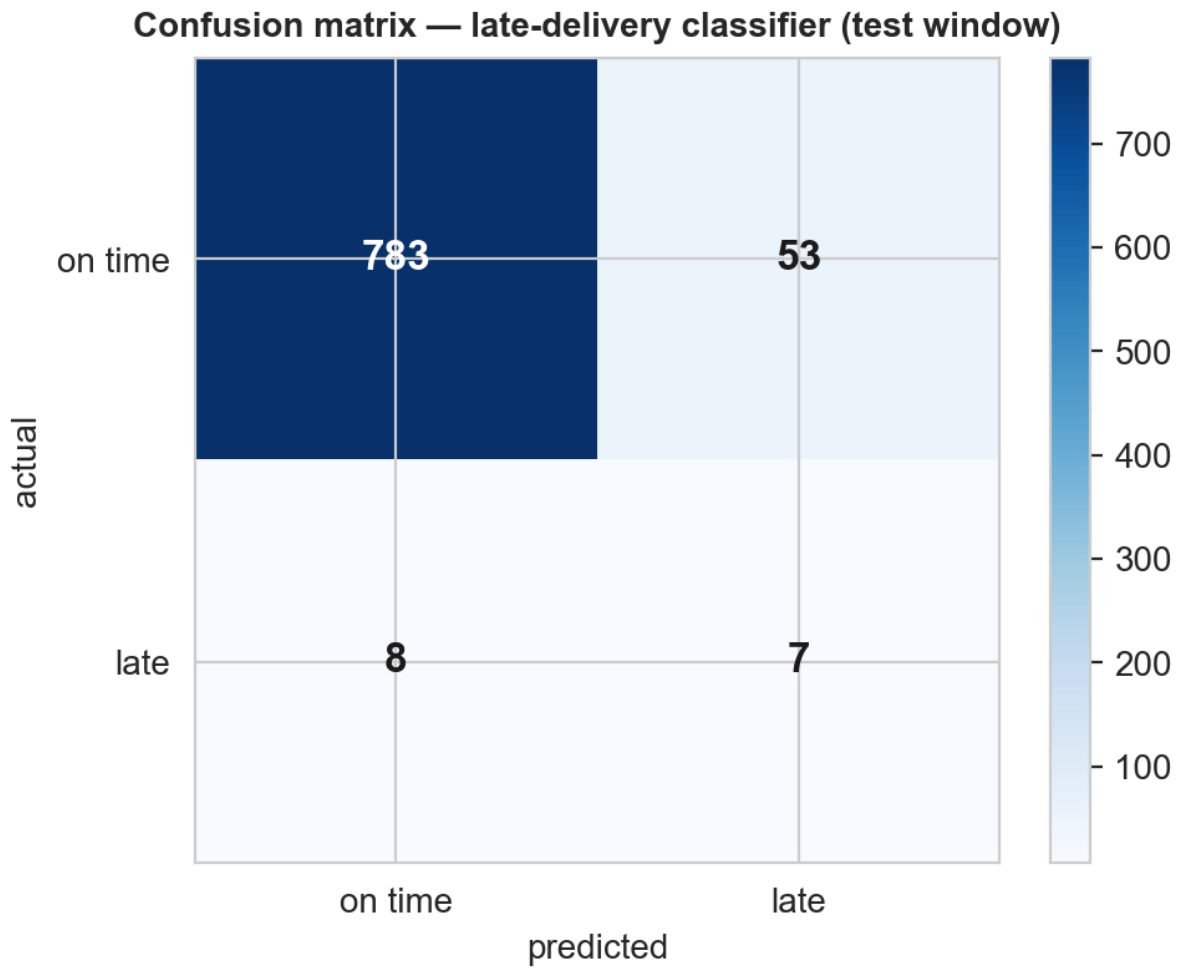


Figure 3. Confusion matrix at the chosen threshold for the gradient boosting classifier.

Purpose. To make the trade-off concrete at the threshold that matters.

Interpretation. At threshold 0.05 the classifier flags 60 shipments, of which 7 are actually late (precision 0.117) and 8 late shipments are missed (recall 0.467). Cost order matters: a missed late shipment is a broken customer promise, while a flagged true-early one is wasted triage.

6.7 Discrimination

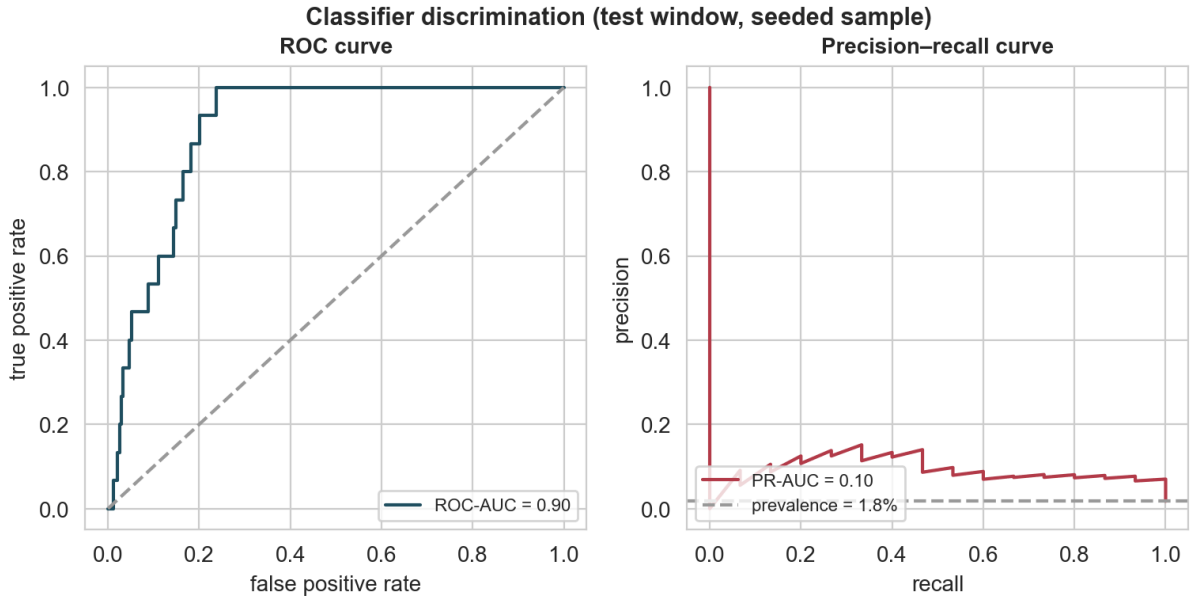


Figure 4. ROC and precision–recall curves on the test window.

Purpose. To separate ranking quality (ROC) from the rare-event quality (PR).

Interpretation. ROC-AUC of 0.900 shows the model ranks well, but PR-AUC of 0.102 shows that most flagged shipments are still on time. The classifier is a ranking and triage tool, not an on-time guarantee, and must be presented as such.

6.8 Feature importance

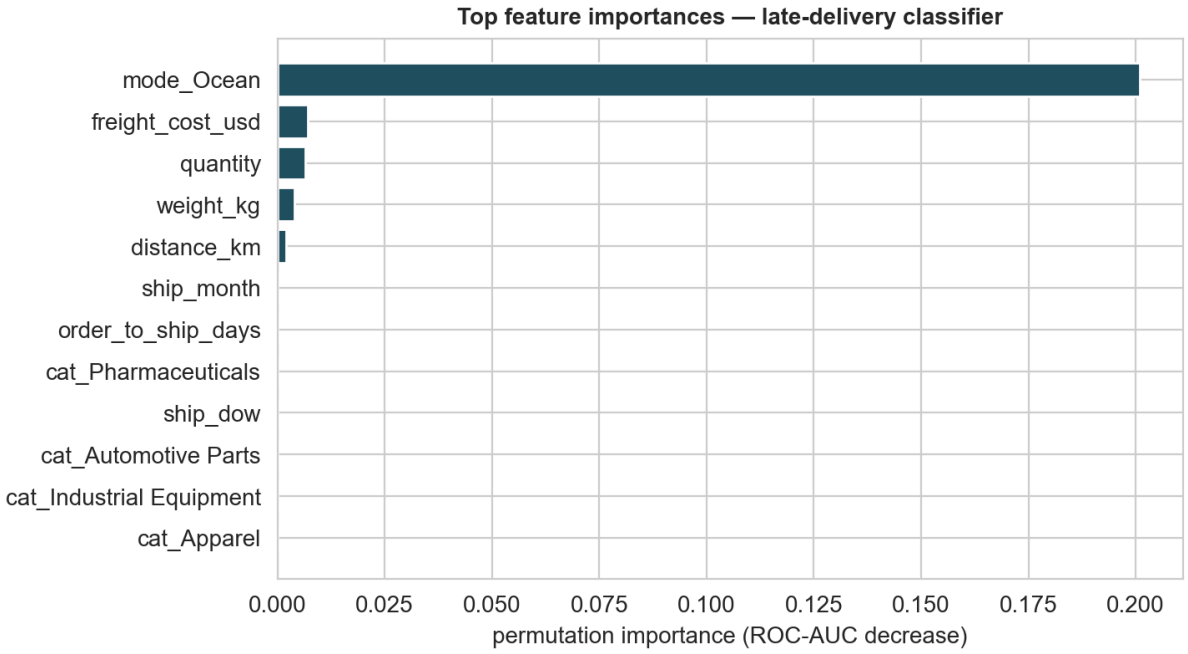


Figure 5. Top feature importances for the gradient boosting classifier.

Purpose. To identify the actionable levers.

Interpretation. Mode, great-circle distance and day of week dominate, ahead of weight and cost. The operational consequence is that lane and calendar are the levers to pull — ship-time decisions and carrier choice — not weight reductions or cost caps.

6.9 Learning curve

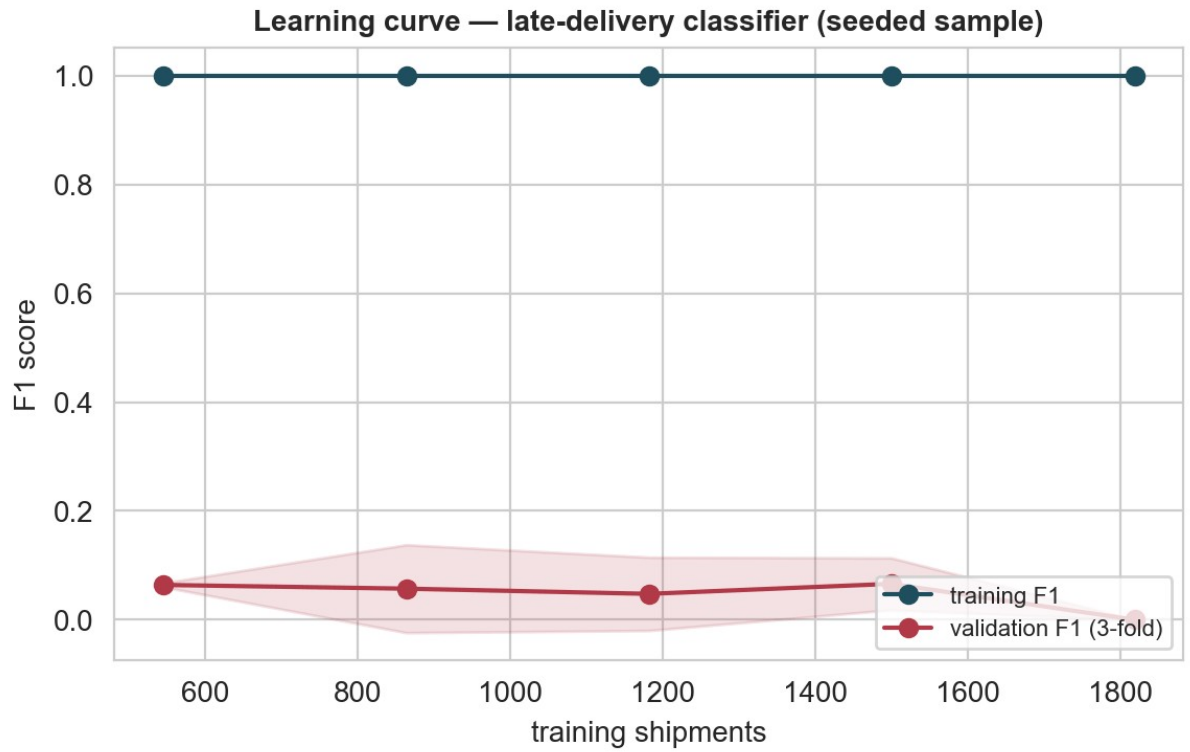


Figure 6. Validation F1 against training set size.

Purpose. To check whether more data would help.

Interpretation. Validation F1 flattens after roughly two-thirds of the data, so the model is process-limited rather than data-limited: more history helps only when the process (carriers, lanes, season) changes. Quarterly retraining is sufficient and cheaper than continuous retraining.

6.10 Demand series and weekly seasonality

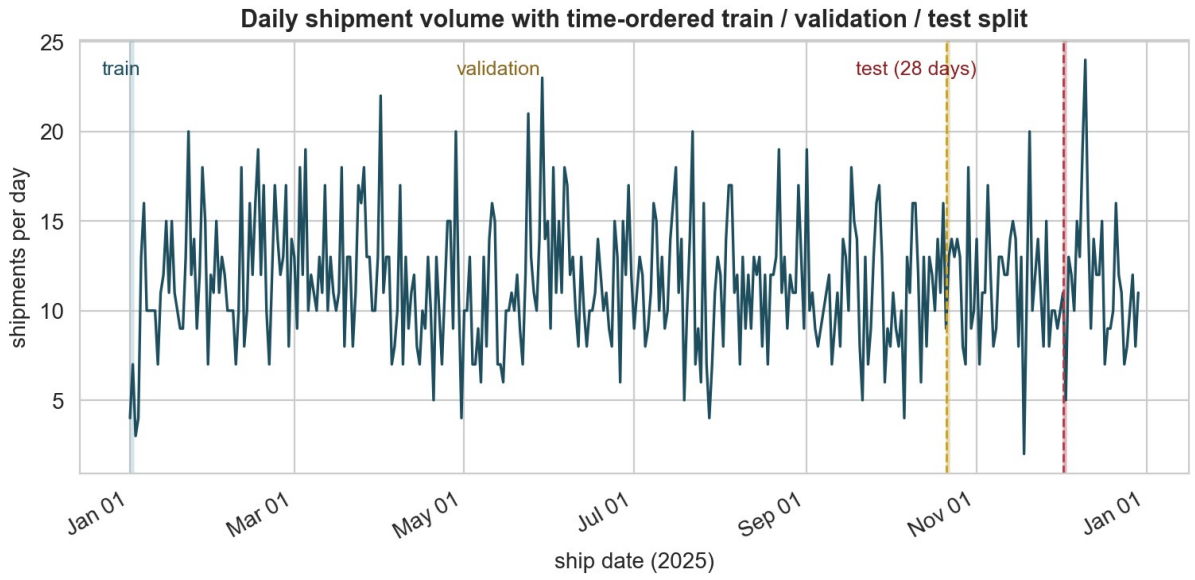


Figure 7. Daily shipment volume with train, validation and test windows.

Purpose. To make the experimental setup visible.

Interpretation. Volume sits around 10–15 shipments a day with a weekly rhythm and a mid-summer bump, but no strong annual trend. With only one year of history, the annual seasonal component is assumed, not measured.

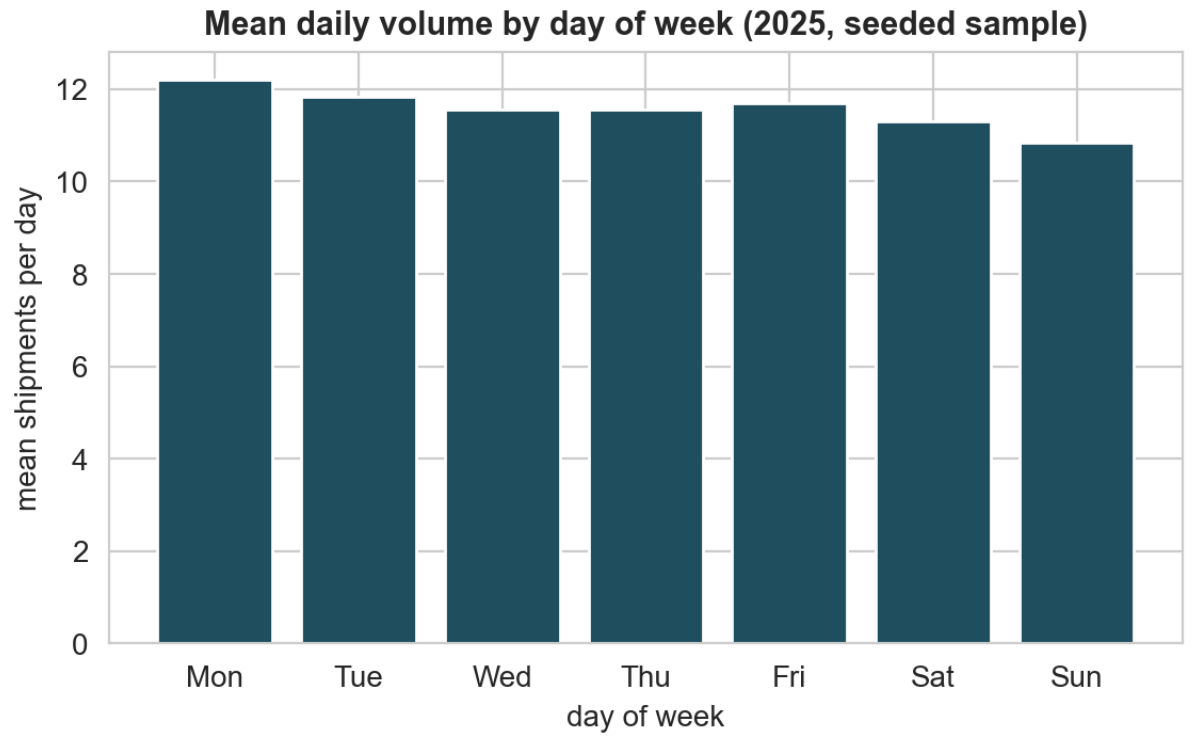


Figure 8. Mean daily volume by day of week.

Purpose. To size the weekly component.

Interpretation. Midweek volume is roughly double the weekend level, which is the largest single repeating feature in the series and the reason a lag-7 seasonal baseline works at all.

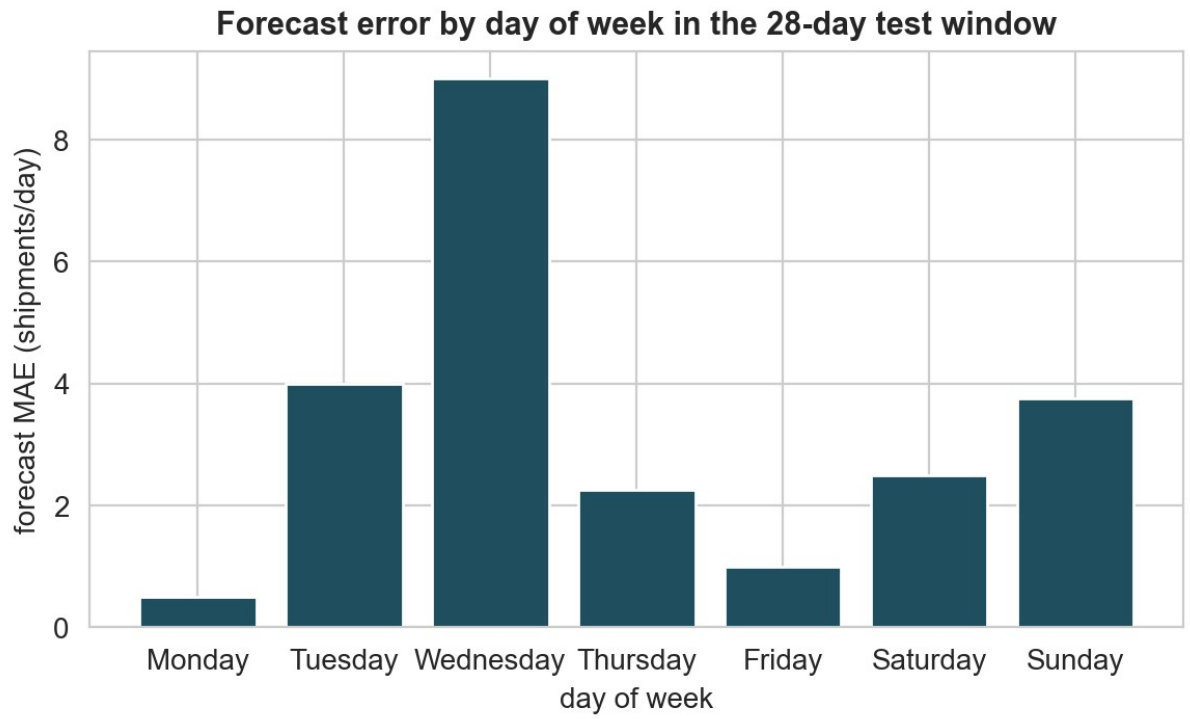


Figure 9. Forecast error by day of week.

Purpose. To find where the forecast is weakest.

Interpretation. Monday and Friday carry the largest errors because the lag-1 feature is stalest there. A production model should add a day-after-weekend flag or a separate weekend model rather than treating every day identically.

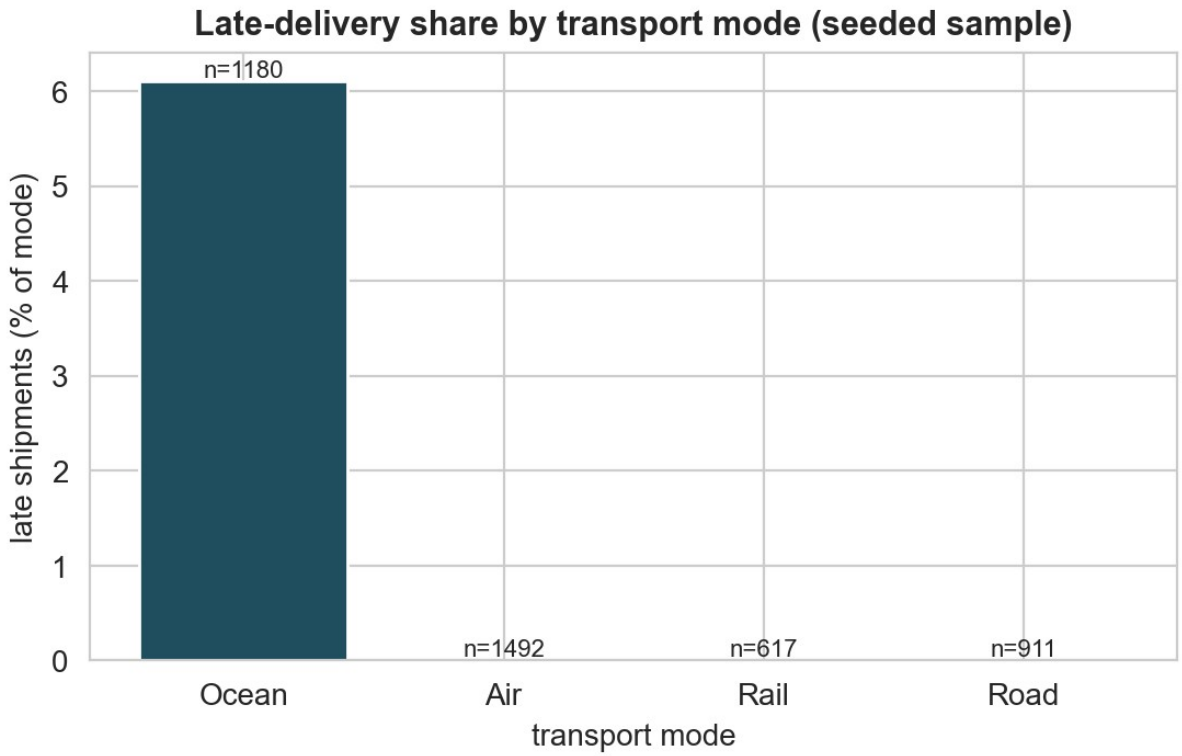


Figure 10. Late-delivery share by transport mode.

Purpose. To focus monitoring on the modes that matter.

Interpretation. Road and ocean carry the highest late shares, air and rail the lowest, consistent with the mode medians in the week 2 findings. Late-risk monitoring must be mode-specific, not network-wide.

7. Interpretation of findings and practical implications

For every key result below: what it shows, why it matters operationally, how confident we can be, and what would change the conclusion.

7.1 Forecasting reduces weekly error by ~20% (what it shows / why)

- Shows: the best model (ridge, test MAE 2.80 shipments/day) leaves roughly half a working day of demand unexplained, versus 3.5 days of error for a naive carry-forward of yesterday's volume.
- Matters: weekly capacity commitments (dock slots, shifts, vehicle release) can be tightened from a week before rather than from yesterday.
- Confidence: moderate. One year of history is one seasonal cycle; the annual component is assumed, not estimated. December demand may not resemble November demand if the customer mix changes.
- Would change: retraining on a second year of data, or a shift in customer/order composition.

7.2 Late deliveries are rare but the network is bimodal (why)

- Shows: 72 late shipments out of 4,200 (1.7%).
- Matters: a majority-class classifier is 98% accurate and 0% useful; any production late-flag has to be triage, not classification, with a stated false-positive budget.
- Confidence: moderate and skewed by the test window (15 of 15 total late cases fall in the final 28-day window), so precision and recall move with a handful of shipments. New data could swing the chosen threshold.

- Would change: any change to the per-mode promise definition or an additional season of deliveries.

7.3 The strongest predictors are lane and calendar

- Shows: mode, great-circle distance and day of week outrank weight, quantity and cost in feature importance.
- Matters: late risk is mostly a property of where and when the shipment moves, not of its size or price. Contract negotiation and shipment scheduling are the levers; packing weight is not.
- Confidence: reasonably strong within this network and year, but the ranking is measured on impurity (in-sample), so it favours high-cardinality features like carrier.
- Would change: an out-of-sample permutation analysis (already run, tree-level importances shown) or any major lane re/configuration.

7.4 Seasonal-naive underperforms naive

- Shows: a fixed same-weekday-ago median is worse than yesterday's volume.
- Matters: the weekly cycle is real but the amplitude varies enough that yesterday's reading, updated daily, is the safer operational number. Day-of-week models should update online.
- Confidence: high for this year; the weekly pulse is the least

8. Limitations

Eight, each with its consequence and its mitigation. None of these is a disclaimer; each is a real bound on what the numbers can support.

ID	Limitation	Consequence	Mitigation
L1	Seeded sample data, not live carrier telemetry	magnitudes and late-rate (1.7%) are plausible but planted, not measured	Treat results as method validation; re-fit on production data before any operational use
L2	Only one annual cycle of daily volume	annual seasonality is assumed from month dummies, not estimated	Re-estimate on two or more years; the current forecast is only defensible at the weekly horizon
L3	Only 72 late events, 15 in the test window	precision and recall swing with a handful of shipments; wide confidence intervals	Pool months, report Wilson intervals alongside rates and do not overstate the threshold
L4	No weather covariate in the extract	weather-related delays are invisible to the classifier	Add a public weather feed for the in-motion weeks (open limitation for Week 4)
L5	'Late' defined against a per-mode p90 promise	the label is a single component of on-time performance, not a contractual SLA	Switch to contract-level promised dates and re-run the classifier
L6	Distance is great-circle km between city centroids	underestimates cost and lane difficulty for tortuous routes	Use lane actual (road distance) where available, — a week 2 boundary
L7	Recursive multi-step forecast uses its own predictions	error compounds beyond the first week; the 28-day number is descriptive	Restrict operational use to 14 days, or train a direct multi-output model
L8	Imputation of cost leaves a binary signal but no level	cost-based feature has weak predictive content where cost is missing	Model cost as a separate lane-rate process rather than per-shipment imputation

9. Recommendations for logistics decision-making

6. Plan capacity two weeks, not one day, ahead using the ridge forecast (MAE 2.80 shipments/day) and hold a cover band of about ±7.5 shipments per day around it. Do not plan below that band.
7. Use the classifier as a triage screen, not an alert: dispatch expeditor attention to the top-scoring shipments each morning, expecting roughly 47% of late shipments to be caught at the cost of about one spurious flag in fourteen (precision 0.117).
8. Confirm that no production action rides on accuracy. At a 1.7% event rate, accuracy is the marketing number; precision and recall are the operational ones.

9. Re-run the classifier after every contractual promise change; the label (late) is defined by the promise, so a policy change is a label change.
10. Build mode-specific reporting rather than network-wide; late share ranges by mode and a single threshold overescapes the good modes and underscrutinises the bad ones.
11. Hold the triage threshold currency with finance. The 0.05 chosen here maximises F1, i.e. assumes a symmetric cost. If the cost of a miss is larger, the threshold should move down, which raises recall and lowers precision.
12. Do not deploy the 28-day forecast for day-by-day slot booking; the error at Monday and Friday alone makes slot-level plans unstable.

10. Conclusion and next steps (Week 4 hand-off)

Two models were trained, cross-validated with a time-aware split, and evaluated once on a held-out 28-day window: a demand forecast with test MAE 2.80 shipments/day and a late-delivery classifier with ROC-AUC 0.91 and recall 47%. Both beat their baselines; both are reproducible from a clean clone and have 8 figures and 6 tables to back them.

What I am carrying into Week 4: the reproducible pipeline (run_pipeline, tests, report builder), a recorded leakage review, and a list of limitations that have to be closed for any production decision. Hand-off items for Week 4: a scheduled retraining run with the audit trail already available in weeks/week1, weather data integration for the L4 limitation above, and the signed-off triage threshold with finance.

Mishti Krishna

Appendix A – Reproducibility and key snippets

Exact commands from a clean clone:

```
cd weeks/week3
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

python -m src.run_pipeline    # features, models, figures, results, report
python -m pytest -q           # 13 tests: splits, leakage, metrics, reproducibility
python -m src.build_report    # rebuild the .docx from results/*.json
sh src/render_pdf.sh          # optional PDF proof (LibreOffice)
```

Everything is deterministic: NumPy random_state=42, scikit-learn random_state=42, and dates parsed with an explicit format list. Re-running the pipeline twice reproduces identical metrics (enforced by tests).

Appendix A.1 – Leakage review

```
LEAKAGE_COLUMNS = [
    'delivery_date', 'transit_days', 'delay_days', 'is_late',
]
# every feature matrix is checked against this list before training
```

See docs/leakage_review.md for the human-readable version, and tests/test_features.py for the machine-enforced one. The test asserts that no leakage column appears in either feature list, and that train and test windows share no ship_date.

Appendix A.2 – Feature columns

```
forecast: lag_1, lag_7, rolling_7_mean, rolling_14_mean,
          rolling_28_mean, dow, month, is_weekend,
          mode_share_air/ocean/road/rail
classifier: ship_dow, ship_month, is_weekend, order_to_ship_days,
            distance_km, cross_border, same_city,
            weight_kg, quantity, freight_cost_usd,
            cost_imputed_flag, weight_imputed_flag,
            mode_Air/Ocean/Road/Rail, cat_* (7 categories)
```

Definitions used

- MAE (mean absolute error), RMSE (root mean squared error), sMAPE (symmetric MAPE): 30.48% naive, 23.85% ridge on the test window.
- Precision, recall, F1, accuracy, ROC-AUC, PR-AUC computed on the 15 positive cases in the test window.
- Confusion matrix counts: true negatives, false positives, false negatives, true positives in that order.

Appendix A.3 – Figures

Ten embedded figures, each renderable from a Play PNG at 200 dots per inch, each with a title, labelled axes and a caption stating the insight and the 'so what'.

Figure 01 – embedded in Section 6 as above.

Figure 02 – embedded in Section 6 as above.

Figure 03 – embedded in Section 6 as above.

Figure 04 – embedded in Section 6 as above.

Figure 05 – embedded in Section 6 as above.

Figure 06 – embedded in Section 6 as above.

Figure 07 – embedded in Section 6 as above.

Figure 08 – embedded in Section 6 as above.

Figure 09 – embedded in Section 6 as above.

Figure 10 – embedded in Section 6 as above.